

IBA KARACHI

Course: Object Oriented Programming

Semester: Spring 2026

Project title: Teen Patti Simulator (Console-Based Card Game)

GROUP MEMBERS:

- **Ajeet Singh (ERP: 33506)**
- **Ayesha Khalid (ERP: 30368)**

1. Introduction & Objectives

We are developing a fully Graphical User Interface (GUI) based Teen Patti (Indian Poker) Simulator for our semester project. The main objective is to deeply integrate Object-Oriented Programming (OOP) concepts such as polymorphism, inheritance, and encapsulation with modern event-driven programming and graphics rendering. By choosing a GUI-based card game, we will demonstrate our ability to bind complex backend logic (probability algorithms, hand evaluation, and betting systems) to a visual frontend using the SFML graphics library.

2. System Description

The game will run in a dedicated graphical window displaying a 2D poker table, complete with interactive buttons and visual card sprites. Each player will be dealt 3 cards visually. Using the mouse, the human player will interact with on-screen buttons to "Bet," "See," "Blind," or "Fold." The game engine will dynamically update the visual chip stacks and the betting pot in real-time. The system will evaluate the strength of the players' hands (Trail, Pure Sequence, Color, Pair) to determine the winner. Progress, such as the player's total chip bankroll, will be saved using file handling to allow continuous play across multiple sessions.

3. Technical Approach (OOP & Data Structures)

We will strictly follow the technical requirements provided in the project document. Here is how we plan to use them:

OOP Concepts:

- **Classes & Objects:** The game relies on core entities: Player, Card, Deck, and a UIManager.

- **Inheritance:** We will have a parent class Player and derived classes like HumanPlayer and AIPlayer.
- **Polymorphism & Abstract Classes:** We will implement an IRenderable interface with a pure virtual draw(RenderWindow& window) function. All visual elements (Cards, Buttons, Chips) will inherit this to easily render the game frame by frame.
- **Encapsulation:** Visual assets (textures, sprites) and game data (chip balance) will be private, accessed only through secure setter/getter methods.
- **Operator Overloading:** We will overload the > operator to easily compare two hands to determine the winner (e.g., if (player1 > player2)).

Data Structures:

- **Queue:** A Queue will manage the "Turn Order" of the players sitting at the graphical table, dictating who has access to the UI controls at any given moment.
- **Vector (STL):** The deck of 52 cards and the visual sprite lists will be managed using `std::vector` for dynamic memory allocation.

Algorithms & Advanced Features:

- **Sorting:** Hand sorting algorithms will be applied to organize and evaluate sequence patterns (e.g., automatically sorting a 4-2-3 hand into 2-3-4 for evaluation).
- **Exception Handling:** Robust **try-catch** blocks will ensure the game does not crash if a visual asset (like a card image file) fails to load.

4. Key Modules & Classes:

1. **GameEngine:** The backend logic that manages the betting pot, turn queues, and game rules.
2. **UIManager:** The frontend interface that uses SFML to draw the table, render text (fonts), load card textures, and detect mouse clicks.

3. **Deck:** Generates the 52 **Card** objects and handles shuffling algorithms.
4. **Player (Abstract):** Parent class for participants
5. **HumanPlayer:** Driven by mouse-click events on the GUI.
6. **AIPlayer:** Driven by internal logic to bet or fold.

5. Division of Work

To ensure we finish on time, we have divided the work as follows:

Ajeet Singh (ERP: 33506)

- **Game Engine & Turn Logic:** Setting up the main game loop and the Queue data structure to manage player turns.
- **Deck Management:** Implementing the **std::vector** to hold the backend card logic and shuffling.
- **SFML UI Integration:** Creating the main **UIManager** class to open the window, handle the frame-rate, and detect mouse clicks on buttons.
- **File Handling:** Writing the code to save and load the player's chip bankroll.

Ayesha Khalid (ERP: 30368)

- **OOP Hierarchy:** Implementing the abstract **Player** class, **HumanPlayer**, and **AIPlayer** classes.
- **Graphical Assets & Card Design:** Linking the backend **Card** class to SFML Textures and Sprites to visually render the cards on screen.
- **Hand Evaluation (Algorithms):** Writing the sorting and evaluation logic to determine winning hands (Pairs, Sequences, Trails).
- **Operator Overloading:** Overloading comparison operators to determine the winner.

6. Conclusion

This GUI-based project covers all mandatory OOP topics such as polymorphism, inheritance, and encapsulation while utilizing data structures like Vectors and Queues. By integrating the SFML library, we are pushing beyond standard console applications to create a modern, event-driven software architecture that completely satisfies the course learning outcomes.